



STUDENT GUIDE

Session 01

End-to-End MLOps & Production Readiness

BITS Pilani · Professional AI/ML Programme · Module 05



26 Interactive Cells



3 Built-in Quizzes



Local Simulation Mode



AWS Lab Ready

Your Learning Toolkit



STUDY MODE

Read slides + run notebook with
`RUN_AWS = False`



LAB MODE

Set `RUN_AWS = True` inside
approved BITS sandbox



REVIEW MODE

Revisit concept slides +
retake quizzes after session

Navigation Tips

- ① Every slide has a matching notebook cell — look for the  icon
- ② Concept slides → Quiz slides → Implementation slides → Practice slides
- ③ Speaker notes (visible to presenter) have teaching tips and timing



Why Your Notebook Isn't Enough



Home Cook

(Your Notebook)

- You control ingredients & timing
- Works perfectly on YOUR machine
- No documentation needed
- You fix it when it breaks
- One portion for yourself
- No one else can recreate it



Scale
up!



Restaurant Chain

(Production ML)

- Standardized, documented recipes
- Works identically everywhere
- Full audit & compliance logs
- Automated monitoring + alerts
- 10,000 portions simultaneously
- Any new chef can reproduce it



Key insight: Production ML is not just 'running the model' — it's a sociotechnical system.

What Goes Wrong Without MLOps



Bias

Amazon Hiring (2018)

Problem: Model trained on historical hiring data learned to penalise résumés mentioning 'women's chess club'

MLOps fix: Data validation + fairness audits are non-negotiable pre-deployment gates



Edge Case

Tesla Autopilot (2016)

Problem: Could not distinguish a white truck against a bright sky — an edge case not in training data

MLOps fix: Out-of-distribution (OOD) detection and adversarial testing must be part of the release gate



Data Drift

Zillow iBuying (2021)

Problem: Home price model miscalculated during COVID market shifts — \$569M loss, product shut down

MLOps fix: Model drift monitoring: when the world changes, the model must be retrained or retired

Same Family, Different Challenges

Dimension	DevOps (Software)	MLOps (ML Systems)
Code behaviour	Deterministic — same input → same output always	Probabilistic — output depends on training data
Testing	Unit tests, integration tests	Accuracy metrics + bias tests + drift detection
Versioning	Git for code only	Git + DVC for code + data + models
Deployment	Ship new version of app	Ship new model + serving infra together
Monitoring	CPU, memory, latency	Also: accuracy, data drift, feature distribution
Failure mode	App crashes → obvious error	Model degrades silently → insidious, no alert

 The most dangerous column is the last row: 'silent degradation' — a model can fail for weeks with no error log.

Where Is Your Team Right Now?



★ This course targets the Level 1 → Level 3 transition using AWS managed services.



QUICK CHECK

Think before you advance →

Q1: A retail company deploys a fraud-detection model. Six months later, fraud patterns change but the model isn't retrained. What type of failure is this?

A) System crash B) Data drift / concept drift C) IAM misconfiguration D) Docker failure

✓ Answer: B — The world changed but the model didn't. This is data drift / concept drift.

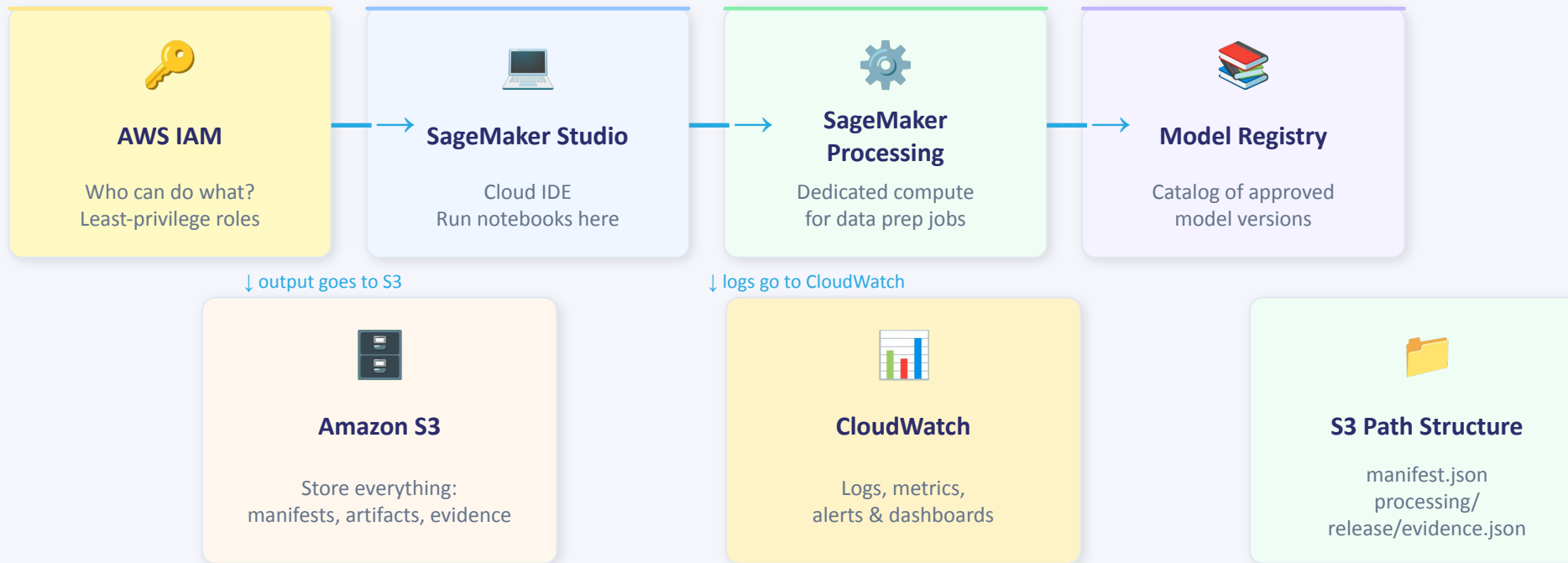
Q2: Which of these is a key difference between MLOps and DevOps?

A) MLOps uses Python, DevOps uses Java B) MLOps also versions training data and monitors model accuracy C) MLOps replaces DevOps entirely D) DevOps is harder than MLOps

✓ Answer: B — MLOps adds data versioning, model monitoring, and continuous training to the DevOps toolkit.

Cover the green boxes with a sticky note before class — reveal after 60 seconds of student thinking time.

Services We Use & Why



The Three Pillars of Production ML



Reliability

"The system does what it's supposed to do, even when things go wrong."

- Predictions served even if one data source fails (fallback logic)
- Graceful degradation: serve cached result if model crashes
- SLA: 99.9% uptime = max 8.7 hrs downtime/year

 SageMaker multi-AZ endpoints, CloudWatch alarms



Scalability

"The system handles 10× traffic without code changes."

- Diwali sale: prediction traffic spikes 50× — auto-scaling handles it
- Training on 1TB runs in same time as 10GB (distributed)
- Feature computation scales horizontally

 SageMaker auto-scaling, S3 unlimited storage



Security

"Only the right people and systems can access what they need."

- Model weights encrypted at rest (AES-256)
- Only inference role can read model artifact (IAM least-privilege)
- All API calls logged in CloudTrail for audit

 AWS IAM roles, S3 server-side encryption, CloudTrail

What We Build & Why Each Step Matters

1



Write the Run Manifest

A JSON record of what we're building: services, controls, learning objectives



Even if the run fails halfway, we have an audit trail of intent. Think: construction permit before any digging starts.

```
manifest = {  
  'project': PROJECT,  
  'run_id': str(uuid.uuid4()),  
  'controls': ['versioned S3  
evidence', ...]  
}
```

2



Submit Processing Job

A containerised data processing job on ml.m5.large with a 30-min auto-stop



Processing jobs run on dedicated compute — your Studio notebook isn't slowed down, and the job survives if you close your laptop.

```
sm.create_processing_job(  
    ProcessingJobName=job_name,  
  
    StoppingCondition={'MaxRuntimeInSeconds': 1800}  
)
```

3



Create Model Registry Group

A named 'folder' in SageMaker Model Registry for all versions of this model

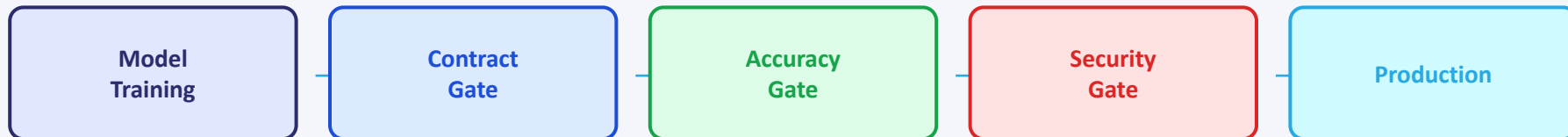


Without a registry, models are .pkl files floating in S3 with no metadata. The registry gives each version approval status, metrics, and owner.

```
sm.create_model_package_group(  
  
    ModelPackageGroupName=registry_group  
)
```

Proving a Model is Ready for Production

Release Gate Pipeline



If ANY gate fails → model is blocked. Back to training.

Evidence Bundle — What Gets Written to S3

✓ artifact_uri	Where the model lives — ops team finds it here
✓ schema reviewed	Input/output contract matches training and serving
✓ logs available	CloudWatch log group confirmed — debugging is possible
✓ failure path defined	What happens if the model throws an exception?
✓ rollback documented	How to undo this deployment in < 10 minutes

Who Else Cares About Your Model?

DevOps / Infra

"We want ML to work like any other service."

- Docker image with API endpoint
- Health check URLs (/health, /metrics)
- CPU/RAM/GPU requirements in writing

QA / Testing

"We need to be able to test your model."

- Test dataset with known expected outputs
- Acceptance threshold (e.g. $F1 > 0.85$)
- Staging environment before production

Security / Compliance

"We are responsible if this causes harm."

- PII audit: what personal data is processed?
- Model card: training data provenance + known biases
- Incident response: shutdown in < 10 minutes

Product Team

"We care about outcomes, not just accuracy."

- Business metric tracking (not just F1 score)
- A/B test results before full rollout
- Interpretability: why did the model predict X?



MLOps practitioners are translators — they speak data science to engineers, engineering to product, both to security.



IMPLEMENTATION CHECK

Notebook → Production

Q1: Why do we write the run manifest BEFORE creating any AWS resources?

A) It's required by the SageMaker API B) So there's an audit trail even if the run fails halfway C) To pre-calculate costs D) To configure IAM roles

✓ B — If the job fails at Step 3, we still have a record of everything we intended to create. This is how enterprise audits work.

Q2: A SageMaker Processing job has StoppingCondition: { MaxRuntimeInSeconds: 1800 }. What happens at 30 minutes?

A) The job pauses and waits for manual resume B) AWS sends an email notification C) The job is automatically stopped — protecting against runaway costs D) The job scales up to a larger instance

✓ C — Without this, a bug could keep a job running for hours or days, generating unexpected costs. Always set a stopping condition.

Q3: Which stakeholder would first raise concerns if your model outputs a customer's home address?

A) DevOps team B) QA team C) Product team D) Security / Compliance team

✓ D — PII leakage is a compliance and legal issue. Security teams handle data protection regulations (GDPR, DPDPA in India).

Local Simulation: No AWS Required!

Section 6 of the notebook gives you a full local MLOps pipeline to experiment with:



Schema Mismatch

Exercise 1: Trigger the Data Drift Alert

Change: Set `inject_drift = True`

Observe: The pipeline stops at Step 1 — no training, no registry. This mirrors real MLOps: never train on bad data.



Gate Calibration

Exercise 2: Set an Impossible Gate Threshold

Change: Set `ACCURACY_GATE = 0.99`

Observe: The model fails Step 3 even with a good model. Adjust down to 0.80 to see it pass. Feel the difference between aspirational and realistic gates.



Schema Evolution

Exercise 3: Add a Feature the Data Doesn't Have

Change: Set `feature_schema = [..., 'city']`

Observe: Since incoming data has no 'city' column, drift is detected in Step 1. This simulates a real incident where a data team silently changes schema.

 *Reflection: What would you add to handle each failure mode automatically in production?*



Session 01 Summary

What You Covered Today

Production Gap

Notebooks need reproducibility, monitoring & governance to become production systems

Maturity Levels

Level 0→1→2→3: this course targets the most impactful transitions

Manifest First

Write audit trail before creating any resource — always

Stakeholders

DevOps, QA, Security, Product all have different needs from your model

MLOps vs DevOps

MLOps adds data versioning, model monitoring, and continuous training

AWS Architecture

IAM → Studio → Processing → Registry → S3 + CloudWatch

Evidence Bundle

Links model artifact to its validation history; gates must pass

Cleanup Always

Tag resources. Set expiry. Delete after the session. No exceptions.

Next: Session 02 — Pipeline Orchestration with Apache Airflow →